

Конспект лекции 11: ООП: продолжение

Преподаватель: Сергей Шершаков

1. Структуры и пользовательские типы данных

- **Понятие объекта:** Каждый объект в C++ имеет тип, который строго определяет его размер в памяти, внутреннюю структуру и поддерживаемые операции. Например, базовый тип `int` — это обычно 32-битное целое число из 4 байт, организованных по принципу *little-endian* (младшие байты идут первыми).
- **Создание типа через `struct`:** Ключевое слово `struct` позволяет объявить собственный составной тип данных и дать ему имя. Фигурная скобка с точкой запятой в конце завершают это объявление.
- **Пространство состояний:** Это множество всех различных значений, которые может принимать объект определенного типа. Если 32-битный `int` может принимать 2^{32} значений, то структура из двух таких полей имеет пространство состояний в $2^{32} \times 2^{32} = 2^{64}$ комбинаций.
- **Семантика типа:** Пользовательский тип данных определяет не только структуру (пространство состояний), но и пользовательский набор операций, задающий его семантику.

```
struct Vector2d {  
    double x;  
    double y;  
};
```

- **Передача и возврат по ссылке:** Объекты структур можно передавать в функции по ссылке (например, `void multByScalar(Vector2d& v, double z)`), чтобы изменять их исходное состояние без лишнего копирования, а также возвращать по ссылке для дальнейшего использования.

2. Перегрузка оператора вывода `operator<<`

- **Обучение компилятора:** Чтобы стандартный поток вывода (например, `std::cout`) умел работать с созданными пользователем структурами, необходимо перегрузить оператор `<<` для типа `std::ostream`.
- **Сигнатура перегрузки:**

```
std::ostream& operator<<(std::ostream& s, const Vector2d& v) {  
    s << '(' << v.x << ", " << v.y << ')';  
}
```

```
    return s;
}
```

- **Почему возвращается ссылка на поток?** Оператор должен возвращать поток по ссылке (`std::ostream&`), чтобы поддерживать последовательные цепочки вызовов (например, `std::cout << v1 << ' ';`).

3. Проблема несогласованности данных (Data Inconsistency)

- **Суть проблемы:** Если мы захотим оптимизировать вычисления и закешировать длину вектора в отдельном поле `length` (чтобы не пересчитывать её каждый раз), возникнет проблема синхронизации данных при изменении координат `x` или `y`.
- **Флаг инициализации:** Отрицательное значение в поле `length` может выступать индикатором того, что длина еще ни разу не вычислялась.
- **Решения проблемы:**
 1. Полностью запретить изменение координат `x` и `y` извне после создания объекта.
 2. Сделать так, чтобы любое изменение координат автоматически инвалидировало сохраненное значение `length`.

4. От внешних функций к методам структуры

- **Объединение данных и поведения:** Вместо передачи объекта структуры во внешние функции в качестве параметра, логичнее связать данные и функции обработки воедино. Внутри методов структуры сам объект доступен неявно.
- **Реализация методов:** При написании тела метода в `.cpp` файле используется оператор разрешения области видимости `::` для указания принадлежности к структуре.

```
Vector2d& Vector2d::multByScalarEnh(double z) {
    x *= z; // Имя текущего объекта подразумевается неявно
    y *= z;
    return *this; // Возврат самого себя
}
```

5. Ключевое слово `this`

- **Определение:** `this` — это встроенный указатель на текущий экземпляр (инстанс) объекта, внутри метода которого выполняется код.
- **Разыменование:** Используется, когда требуется явный возврат или адресация текущего объекта (например, `return *this;` для возврата объекта по ссылке).
- **Правило избыточности:** В C++ использование `this->` внутри методов для доступа к собственным полям чаще всего избыточно (в отличие от обязательного `self` в Python). Правило лекции: *никогда не пишите `this`, кроме случаев, когда это действительно необходимо.*

6. Инкапсуляция и сокрытие данных

- **Концепция:** Инкапсуляция — это сборка деталей реализации воедино и отделение их от публичной абстракции. Соккрытие данных (перенос полей в закрытую секцию) является частным случаем инкапсуляции.
- **Модификаторы доступа:**
 - **private** — делает члены структуры доступными только изнутри самой структуры (внутренняя реализация).
 - **public** — открывает члены для внешнего кода, формируя публичный интерфейс взаимодействия (абстракцию).
- **Оформление кода (Code Style):** Согласно правилам хорошего тона программирования, скрытые непубличные поля принято именовать с подчеркиванием (например, `length_` или `_length`).

7. Конструкторы классов

- **Определение:** Конструктор — это специальный метод класса/структуры, который вызывается автоматически в момент создания объекта в памяти.
- **Особенности:**
 - Имеет имя, в точности совпадающее с именем класса/структуры (для класса `Foo` конструктор будет `Foo::Foo()`).
 - **Не имеет возвращаемого значения** (даже `void` указывать нельзя).
- **Виды конструкторов:** Конструктор по умолчанию (без параметров), конструкторы инициализации (с произвольными параметрами), специальные конструкторы (конструктор копирования, конструктор перемещения).

8. Список инициализации членов (Member Initializer List)

- **Синтаксис:** Представляет собой разделенный запятыми список инициализаторов полей, который указывается после двоеточия перед открывающей фигурной скобкой тела конструктора.
- **Важность использования:** Поля класса *должны* инициализироваться именно через список инициализации, а не путем повторного присваивания значений уже внутри тела конструктора.

```
Vector2d::Vector2d(double init_x, double init_y)
    : x_(init_x), y_(init_y), length_(-1.0) // Список инициализации
{
    // Тело конструктора остается пустым
}
```

| 9. В чем разница между *struct* и *class*?

Различия сводятся исключительно к правилам видимости по умолчанию:

- В структуре (*struct*) все члены и тип наследования по умолчанию являются публичными (*public*).
- В классе (*class*) все члены и тип наследования по умолчанию являются приватными (*private*).

С технической точки зрения в C++ между ними **нет больше никаких различий**.